

Python Sets

A set is a collection of unique data, meaning that elements within a set cannot be duplicated. For instance, if we need to store information about student IDs, a set is suitable since student IDs cannot have duplicates.

Create a Set in Python

In Python, we create sets by placing all the elements inside curly braces {}, separated by commas.

A set can have any number of items and they may be of different types (integer, float, [tuple](#), [string](#), etc.). But a set cannot have mutable elements like [lists](#), sets or [dictionaries](#) as its elements.

create a set of integer type

```
student_id = {112, 114, 116, 118, 115}
```

```
print('Student ID:', student_id)
```

create a set of string type

```
vowel_letters = {'a', 'e', 'i', 'o', 'u'}
```

```
print('Vowel Letters:', vowel_letters)
```

create a set of mixed data types

```
mixed_set = {'Hello', 101, -2, 'Bye'}
```

```
print('Set of mixed data types:', mixed_set)
```

Output

```
Student ID: {112, 114, 115, 116, 118}
```

```
Vowel Letters: {'u', 'a', 'e', 'i', 'o'}
```

```
Set of mixed data types: {'Hello', 'Bye', 101, -2}
```

Note: When you run this code, you might get output in a different order. This is because the set has no particular order.

Create an Empty Set in Python

Creating an empty set is a bit tricky. Empty curly braces {} will make an empty dictionary in Python.

To make a set without any elements, we use the set() function without any argument.

```
# create an empty set
```

```
empty_set = set()
```

```
# create an empty dictionary
```

```
empty_dictionary = { }
```

```
# check data type of empty_set
```

```
print('Data type of empty_set:', type(empty_set))
```

```
# check data type of dictionary_set
```

```
print('Data type of empty_dictionary:', type(empty_dictionary))
```

Output

```
Data type of empty_set: <class 'set'>
```

```
Data type of empty_dictionary: <class 'dict'>
```

Duplicate Items in a Set

```
numbers = {2, 4, 6, 6, 2, 8}
```

```
print(numbers) # {8, 2, 4, 6}
```

Here, we can see there are no duplicate items in the set as a set cannot contain duplicates.

Add and Update Set Items in Python

Sets are mutable. However, since they are unordered, indexing has no meaning.

We cannot access or change an element of a set using indexing or slicing. The set data type does not support it.

In Python, we use the [add\(\)](#) method to add an item to a set.

```
numbers = {21, 34, 54, 12}
```

```
print('Initial Set:', numbers)
```

```
# using add() method
```

```
numbers.add(32)
```

```
print('Updated Set:', numbers)
```

Output

```
Initial Set: {34, 12, 21, 54}
```

```
Updated Set: {32, 34, 12, 21, 54}
```

Update Python Set

The [update\(\)](#) method is used to update the set with items other collection types (lists, tuples, sets, etc).

```
companies = {'Lacoste', 'Ralph Lauren'}  
tech_companies = ['apple', 'google', 'apple']
```

```
# using update() method  
companies.update(tech_companies)
```

```
print(companies)
```

```
# Output: {'google', 'apple', 'Lacoste', 'Ralph Lauren'}
```

Remove an Element from a Set

We use the [discard\(\)](#) method to remove the specified element from a set

```
languages = {'Swift', 'Java', 'Python'}
```

```
print('Initial Set:', languages)
```

```
# remove 'Java' from a set  
removedValue = languages.discard('Java')
```

```
print('Set after remove():', languages)
```

Output

```
Initial Set: {'Python', 'Swift', 'Java'}
```

```
Set after remove(): {'Python', 'Swift'}
```

Iterate Over a Set in Python

```
fruits = {"Apple", "Peach", "Mango"}
```

```
# for loop to access each fruits
```

```
for fruit in fruits:
```

```
    print(fruit)
```

We can use the [len\(\)](#) method to find the number of elements present in a Set.

```
even_numbers = {2,4,6,8}
```

```
print('Set:',even_numbers)
```

```
# find number of elements
```

```
print('Total Elements:', len(even_numbers))
```

Output

```
Set: {8, 2, 4, 6} Total Elements: 4
```

Python Set Operations

Union of Two Sets

The union of two sets A and B includes all the elements of sets A and B.

We use the | operator or the [union\(\)](#) method to perform the set union operation

first set

A = {1, 3, 5}

second set

B = {0, 2, 4}

perform union operation using |
print('Union using |: ', A | B)

perform union operation using union()
print('Union using union(): ', A.union(B))

Output

Union using |: {0, 1, 2, 3, 4, 5}

Union using union(): {0, 1, 2, 3, 4, 5}

Python Set Operations

Set Intersection

The intersection of two sets A and B include the common elements between set A and B.

In Python, we use the & operator or the [intersection\(\)](#) method to perform the set intersection operation

first set

```
A = {1, 3, 5}
```

second set

```
B = {1, 2, 3}
```

perform intersection operation using &

```
print('Intersection using &:', A & B)
```

perform intersection operation using intersection()

```
print('Intersection using intersection():', A.intersection(B))
```

Output

```
Intersection using &: {1, 3}
```

```
Intersection using intersection(): {1, 3}
```


Python Set Operations

Difference between Two Sets

The difference between two sets A and B include elements of set A that are not present on set B.

We use the - operator or the [difference\(\)](#) method to perform the difference between two sets.

first set

A = {2, 3, 5}

second set

B = {1, 2, 6}

perform difference operation using &

print('Difference using &:', A - B)

perform difference operation using difference()

print('Difference using difference():', A.difference(B))

Output

Difference using &: {3, 5}

Difference using difference(): {3, 5}

Python Set Operations

Set Symmetric Difference

The symmetric difference between two sets A and B includes all elements of A and B without the common elements.

In Python, we use the ^ operator or the [symmetric_difference\(\)](#) method to perform symmetric differences between two sets.

first set

A = {2, 3, 5}

second set

B = {1, 2, 6}

perform difference operation using &
print('using ^:', A ^ B)

using symmetric_difference()
print('using symmetric_difference():', A.symmetric_difference(B))

Output

using ^: {1, 3, 5, 6}

using symmetric_difference(): {1, 3, 5, 6}